05_silver_to_gold (Python) Import Notebook

```
%md
# 05 — Silver to Gold (Dimensional Model + SCD Type 2)
Builds the star-schema Gold layer from Silver:
- Dimensions: dim_customer (SCD2), dim_track, dim_album, dim_artist, dim_genre, dim_media_type, dim_employee,
dim_date
- Facts: fact_sales (invoice line grain), fact_sales_customer_agg (aggregated from fact_sales)
```

05 — Silver to Gold (Dimensional Model + SCD Type 2)

Builds the star-schema Gold layer from Silver:

- Dimensions: dim_customer (SCD2), dim_track, dim_album, dim_artist, dim_genre, dim_media_type, dim_employee, dim_date
- Facts: fact_sales (invoice line grain), fact_sales_customer_agg (aggregated from fact_sales)

```
# Widget parameters
dbutils.widgets.text("catalog_name", "workspace", "Catalog Name")
dbutils.widgets.text("schema_name", "raw_zone", "Schema Name")
dbutils.widgets.text("base_path", "/Volumes/workspace/raw_zone/chinook", "Base Path")
dbutils.widgets.text("table_name", "all", "Table Name")
dbutils.widgets.text("run_id", "", "Run ID (auto-generated if blank)")

catalog = dbutils.widgets.get("catalog_name")
run_id_param = dbutils.widgets.get("run_id")
```

```
from pyspark.sql.functions import (
    col, lit, current_timestamp, monotonically_increasing_id,
    sha2, concat_ws, when, coalesce, min as _min, max as _max,
    count, sum as _sum, to_date, date_format, year, month, dayofmonth,
    dayofweek, quarter, row_number
)
from pyspark.sql.window import Window
from datetime import datetime, date, timedelta
import uuid

try:
    run_id = dbutils.jobs.taskValues.get(taskKey="task_04_bronze_to_silver", key="run_id")
except:
    run_id = run_id_param if run_id_param else str(uuid.uuid4())[:8]

print(f"Run ID: {run_id}")
```

Run ID: 7549bffc

```
%md
## 1. dim_date (Generated — not from source)
```

1. dim_date (Generated — not from source)

```
import pandas as pd

# Generate date range covering all invoice dates
min_max = spark.sql(f"SELECT MIN(InvoiceDate) as min_dt, MAX(InvoiceDate) as max_dt FROM
{catalog}.silver.invoice").collect()[0]
start_date = min_max.min_dt.date() if min_max.min_dt else date(2009, 1, 1)
end_date = min_max.max_dt.date() if min_max.max_dt else date(2025, 12, 31)

# Extend range to full years
start_date = date(start_date.year, 1, 1)
end_date = date(end_date.year, 12, 31)

date_range = pd.date_range(start=start_date, end=end_date, freq='D')
date_data = [{
```

```
        "date_key": int(d.strftime("%Y%m%d")),
        "full_date": d.date(),
        "year": d.year,
        "quarter": (d.month - 1) // 3 + 1,
        "month": d.month,
        "month_name": d.strftime("%B"),
        "day": d.day,
        "day_of_week": d.weekday() + 1,
        "day_name": d.strftime("%A"),
        "is_weekend": 1 if d.weekday() >= 5 else 0
    } for d in date_range]

    df_dim_date = spark.createDataFrame(date_data)
    df_dim_date.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_date")
    print(f"dim_date: {df_dim_date.count()} rows ({start_date} to {end_date})")
```

```
› df_dim_date:pyspark.sql.connect.dataframe.DataFrame = [date_key: long, day: long ... 8 more fields]
dim_date: 1826 rows (2009-01-01 to 2013-12-31)
```

```
%md
## 2. dim_artist
```

2. dim_artist

```
df_artist = spark.sql(f"""
    SELECT
        ArtistId AS artist_key,
        ArtistId AS artist_id,
        Name AS artist_name
    FROM {catalog}.silver.artist
""")
df_artist.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_artist")
print(f"dim_artist: {df_artist.count()} rows")
```

```
› df_artist:pyspark.sql.connect.dataframe.DataFrame = [artist_key: integer, artist_id: integer ... 1 more field]
dim_artist: 275 rows
```

```
%md
## 3. dim_album
```

3. dim_album

```
df_album = spark.sql(f"""
    SELECT
        a.AlbumId AS album_key,
        a.AlbumId AS album_id,
        a.Title AS album_title,
        a.ArtistId AS artist_id,
        ar.Name AS artist_name
    FROM {catalog}.silver.album a
    LEFT JOIN {catalog}.silver.artist ar ON a.ArtistId = ar.ArtistId
""")
df_album.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_album")
print(f"dim_album: {df_album.count()} rows")
```

```
› df_album:pyspark.sql.connect.dataframe.DataFrame = [album_key: integer, album_id: integer ... 3 more fields]
dim_album: 347 rows
```

```
%md
## 4. dim_genre
```

4. dim_genre

```
df_genre = spark.sql(f"""
    SELECT
        GenreId AS genre_key,
        GenreId AS genre_id,
```

```
        Name AS genre_name
    FROM {catalog}.silver.genre
""")
df_genre.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_genre")
print(f"dim_genre: {df_genre.count()} rows")
```

› df_genre:pyspark.sql.connect.dataframe.DataFrame = [genre_key: integer, genre_id: integer ... 1 more field]
dim_genre: 25 rows

```
%md
## 5. dim_media_type
```

5. dim_media_type

```
df_media = spark.sql(f"""
    SELECT
        MediaTypeId AS media_type_key,
        MediaTypeId AS media_type_id,
        Name AS media_type_name
    FROM {catalog}.silver.mediatype
""")
df_media.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_media_type")
print(f"dim_media_type: {df_media.count()} rows")
```

› df_media:pyspark.sql.connect.dataframe.DataFrame = [media_type_key: integer, media_type_id: integer ... 1 more field]
dim_media_type: 5 rows

```
%md
## 6. dim_employee
```

6. dim_employee

```
df_employee = spark.sql(f"""
    SELECT
        EmployeeId AS employee_key,
        EmployeeId AS employee_id,
        FirstName AS first_name,
        LastName AS last_name,
        Title AS title,
        ReportsTo AS reports_to,
        BirthDate AS birth_date,
        HireDate AS hire_date,
        City AS city,
        State AS state,
        Country AS country,
        Email AS email
    FROM {catalog}.silver.employee
""")
df_employee.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_employee")
print(f"dim_employee: {df_employee.count()} rows")
```

› df_employee:pyspark.sql.connect.dataframe.DataFrame = [employee_key: integer, employee_id: integer ... 10 more fields]
dim_employee: 8 rows

```
%md
## 7. dim_track
```

7. dim_track

```
df_track = spark.sql(f"""
    SELECT
        t.TrackId AS track_key,
        t.TrackId AS track_id,
        t.Name AS track_name,
        t.AlbumId AS album_id,
        t.MediaTypeId AS media_type_id,
        t.GenreId AS genre_id,
```

```
        t.Composer AS composer,
        t.Milliseconds AS milliseconds,
        t.Bytes AS bytes,
        CAST(t.UnitPrice AS DECIMAL(10,2)) AS unit_price
    FROM {catalog}.silver.track t
""")
df_track.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_track")
print(f"dim_track: {df_track.count()} rows")
```

```
› df_track:pyspark.sql.connect.dataframe.DataFrame = [track_key: integer, track_id: integer ... 8 more fields]
dim_track: 3503 rows
```

```
%md
## 8. dim_customer — SCD Type 2

**Logic:**
1. Read incoming Silver customer data
2. If dim_customer exists in Gold, compare tracked attributes via hash
3. For changed customers: close old record (end date, is_current=false), insert new active row
4. For new customers: insert with is_current=true
5. For unchanged customers: keep existing row
```

8. dim_customer — SCD Type 2

Logic:

1. Read incoming Silver customer data
2. If dim_customer exists in Gold, compare tracked attributes via hash
3. For changed customers: close old record (end date, is_current=false), insert new active row
4. For new customers: insert with is_current=true
5. For unchanged customers: keep existing row

```
def build_dim_customer_scd2(catalog):
    """Implement SCD Type 2 for dim_customer."""

    # Define tracked attributes for change detection
    tracked_cols = [
        "FirstName", "LastName", "Company", "Address", "City",
        "State", "Country", "PostalCode", "Phone", "Fax",
        "Email", "SupportRepId"
    ]

    # Read incoming Silver data
    df_incoming = spark.sql(f"""
        SELECT
            CustomerId,
            FirstName, LastName, Company, Address, City,
            State, Country, PostalCode, Phone, Fax, Email, SupportRepId
        FROM {catalog}.silver.customer
    """)

    # Add hash of tracked attributes for change comparison
    df_incoming = df_incoming.withColumn(
        "row_hash",
        sha2(concat_ws("||", *[coalesce(col(c).cast("string"), lit("__NULL__")) for c in tracked_cols]), 256)
    )

    current_ts = current_timestamp()
    end_of_time = lit("9999-12-31").cast("date")

    # Check if dim_customer already exists
    table_exists = spark.catalog.tableExists(f"{catalog}.gold.dim_customer")

    if not table_exists:
        # ---- INITIAL LOAD ----
        print("  dim_customer does not exist — performing initial load")
        df_new = df_incoming.select(
            monotonically_increasing_id().alias("customer_key"),
            col("CustomerId").alias("customer_id"),
            col("FirstName").alias("first_name"),
            col("LastName").alias("last_name"),
```

```
            col("Company").alias("company"),
            col("Address").alias("address"),
            col("City").alias("city"),
            col("State").alias("state"),
            col("Country").alias("country"),
            col("PostalCode").alias("postal_code"),
            col("Phone").alias("phone"),
            col("Fax").alias("fax"),
            col("Email").alias("email"),
            col("SupportRepId").alias("support_rep_id"),
            col("row_hash"),
            current_ts.alias("effective_start_date"),
            end_of_time.alias("effective_end_date"),
            lit(True).alias("is_current")
        )
        df_new.write.mode("overwrite").saveAsTable(f"{catalog}.gold.dim_customer")
        return df_new.count(), 0

    else:
        # ---- SCD TYPE 2 MERGE ----
        print("  dim_customer exists — applying SCD Type 2 logic")

        df_existing = spark.read.table(f"{catalog}.gold.dim_customer")
        df_current = df_existing.filter(col("is_current") == True)

        # Get max surrogate key
        max_key = df_existing.agg({"customer_key": "max"}).collect()[0][0] or 0

        # Join incoming with current on natural key
        df_joined = df_incoming.alias("inc").join(
            df_current.alias("cur"),
            col("inc.CustomerId") == col("cur.customer_id"),
            "full_outer"
        )

        # Changed records (hash differs)
        df_changed = df_joined.filter(
            col("cur.customer_id").isNotNull() &
            col("inc.CustomerId").isNotNull() &
            (col("inc.row_hash") != col("cur.row_hash"))
        )

        # New records (not in current Gold)
        df_new_customers = df_joined.filter(col("cur.customer_id").isNull())

        changes_count = df_changed.count()
        new_count = df_new_customers.count()

        print(f"    Changed customers: {changes_count}")
        print(f"    New customers:     {new_count}")

        # 1. All historical (non-current) records stay as-is
        df_historical = df_existing.filter(col("is_current") == False)

        # 2. Unchanged current records stay as-is
        df_keep = df_existing.filter(col("is_current") == True).join(
            df_changed.select(col("cur.customer_id").alias("chg_id")),
            col("customer_id") == col("chg_id"),
            "left_anti"
        )

        # 3. Close old records for changed customers
        df_closed = df_existing.filter(col("is_current") == True).join(
            df_changed.select(col("cur.customer_id").alias("chg_id")),
            col("customer_id") == col("chg_id"),
            "inner"
        ).drop("chg_id").withColumn(
            "effective_end_date", current_ts.cast("date")
        ).withColumn(
            "is_current", lit(False)
        )

        # 4. New active records for changed customers
```

```
        w = Window.orderBy("CustomerId")
        df_changed_new = df_changed.select(
            (row_number().over(w) + max_key).alias("customer_key"),
            col("inc.CustomerId").alias("customer_id"),
            col("inc.FirstName").alias("first_name"),
            col("inc.LastName").alias("last_name"),
            col("inc.Company").alias("company"),
            col("inc.Address").alias("address"),
            col("inc.City").alias("city"),
            col("inc.State").alias("state"),
            col("inc.Country").alias("country"),
            col("inc.PostalCode").alias("postal_code"),
            col("inc.Phone").alias("phone"),
            col("inc.Fax").alias("fax"),
            col("inc.Email").alias("email"),
            col("inc.SupportRepId").alias("support_rep_id"),
            col("inc.row_hash"),
            current_ts.alias("effective_start_date"),
            end_of_time.alias("effective_end_date"),
            lit(True).alias("is_current")
        )

        # 5. Brand new customers
        if new_count > 0:
            w2 = Window.orderBy("CustomerId")
            df_brand_new = df_new_customers.select(
                (row_number().over(w2) + max_key + changes_count).alias("customer_key"),
                col("inc.CustomerId").alias("customer_id"),
                col("inc.FirstName").alias("first_name"),
                col("inc.LastName").alias("last_name"),
                col("inc.Company").alias("company"),
                col("inc.Address").alias("address"),
                col("inc.City").alias("city"),
                col("inc.State").alias("state"),
                col("inc.Country").alias("country"),
                col("inc.PostalCode").alias("postal_code"),
                col("inc.Phone").alias("phone"),
                col("inc.Fax").alias("fax"),
                col("inc.Email").alias("email"),
                col("inc.SupportRepId").alias("support_rep_id"),
                col("inc.row_hash"),
                current_ts.alias("effective_start_date"),
                end_of_time.alias("effective_end_date"),
                lit(True).alias("is_current")
            )
        else:
            df_brand_new = spark.createDataFrame([], df_changed_new.schema)

        # Union all parts
        df_final = (
            df_historical
            .unionByName(df_keep)
            .unionByName(df_closed)
            .unionByName(df_changed_new)
            .unionByName(df_brand_new)
        )

        df_final.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable(f"
{catalog}.gold.dim_customer")
        return df_final.count(), changes_count
```

```
total_customer_rows, scd_changes = build_dim_customer_scd2(catalog)
print(f"\ndim_customer: {total_customer_rows} total rows, {scd_changes} SCD Type 2 changes")
```

```
dim_customer exists — applying SCD Type 2 logic Changed customers: 0 New customers: 0 /databricks/python/lib/python3.11/site-
packages/pyspark/sql/connect/expressions.py:1017: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all
data to a single partition, this can cause serious performance degradation. warnings.warn( dim_customer: 62 total rows, 0 SCD Type 2 changes
```

```
%md
## 9. Verify dim_customer SCD Type 2
```

9. Verify dim_customer SCD Type 2

```
display(spark.sql(f"""
    SELECT customer_key, customer_id, first_name, last_name, city, country, email,
           effective_start_date, effective_end_date, is_current
    FROM {catalog}.gold.dim_customer
    ORDER BY customer_id, effective_start_date
    LIMIT 20
"""))
```

Table

```
# Check for any customer with multiple versions (SCD proof — will show after Version 2 rerun)
print("Customers with multiple SCD2 versions:")
display(spark.sql(f"""
    SELECT customer_id, first_name, last_name, COUNT(*) as versions
    FROM {catalog}.gold.dim_customer
    GROUP BY customer_id, first_name, last_name
    HAVING COUNT(*) > 1
    ORDER BY customer_id
"""))
```

Customers with multiple SCD2 versions:

Table

```
%md
## 10. fact_sales (Invoice Line Grain)
```

10. fact_sales (Invoice Line Grain)

```
df_fact_sales = spark.sql(f"""
    SELECT
        il.InvoiceLineId AS sales_line_key,
        il.InvoiceId AS invoice_id,
        il.TrackId AS track_key,
        i.CustomerId AS customer_id,
        dc.customer_key AS customer_key,
        t.AlbumId AS album_key,
        t.GenreId AS genre_key,
        t.MediaTypeId AS media_type_key,
        CAST(DATE_FORMAT(i.InvoiceDate, 'yyyyMMdd') AS INT) AS date_key,
        i.InvoiceDate AS invoice_date,
        il.Quantity AS quantity,
        CAST(il.UnitPrice AS DECIMAL(10,2)) AS unit_price,
        CAST(il.Quantity * il.UnitPrice AS DECIMAL(10,2)) AS line_amount,
        CAST(i.Total AS DECIMAL(10,2)) AS invoice_total,
        e.EmployeeId AS employee_key
    FROM {catalog}.silver.invoiceline il
    JOIN {catalog}.silver.invoice i ON il.InvoiceId = i.InvoiceId
    JOIN {catalog}.silver.customer c ON i.CustomerId = c.CustomerId
    LEFT JOIN {catalog}.gold.dim_customer dc
        ON c.CustomerId = dc.customer_id AND dc.is_current = true
    LEFT JOIN {catalog}.silver.track t ON il.TrackId = t.TrackId
    LEFT JOIN {catalog}.silver.employee e ON c.SupportRepId = e.EmployeeId
""")

df_fact_sales.write.mode("overwrite").saveAsTable(f"{catalog}.gold.fact_sales")
print(f"fact_sales: {df_fact_sales.count()} rows")
```

```
> df_fact_sales:pyspark.sql.connect.dataframe.DataFrame = [sales_line_key: integer, invoice_id: integer ... 13 more fields]
fact_sales: 2240 rows
```

```
%md
## 11. fact_sales_customer_agg (Aggregated from fact_sales — NOT from Silver)
```

11. fact_sales_customer_agg (Aggregated from fact_sales — NOT from Silver)

```
df_agg = spark.sql(f"""
    SELECT
        fs.customer_key,
        fs.customer_id,
        COUNT(DISTINCT fs.invoice_id)   AS total_invoices,
        COUNT(*)                         AS total_items_purchased,
        SUM(fs.line_amount)              AS total_sales_amount,
        MIN(fs.invoice_date)             AS first_purchase_date,
        MAX(fs.invoice_date)             AS last_purchase_date,
        COUNT(DISTINCT fs.track_key)     AS distinct_tracks_purchased,
        COUNT(DISTINCT fs.genre_key)     AS distinct_genres
    FROM {catalog}.gold.fact_sales fs
    GROUP BY fs.customer_key, fs.customer_id
""")

df_agg.write.mode("overwrite").saveAsTable(f"{catalog}.gold.fact_sales_customer_agg")
print(f"fact_sales_customer_agg: {df_agg.count()} rows")
```

```
> df_agg:pyspark.sql.connect.dataframe.DataFrame = [customer_key: long, customer_id: integer ... 7 more fields]
fact_sales_customer_agg: 59 rows
```

```
%md
## 12. Gold Summary and Validation
```

12. Gold Summary and Validation

```
gold_tables = [
    "dim_date", "dim_artist", "dim_album", "dim_genre",
    "dim_media_type", "dim_employee", "dim_track", "dim_customer",
    "fact_sales", "fact_sales_customer_agg"
]
```

```
print(f"{'='*50}")
print(f"GOLD LAYER SUMMARY — Run ID: {run_id}")
print(f"{'='*50}")

for tbl in gold_tables:
    cnt = spark.sql(f"SELECT COUNT(*) as cnt FROM {catalog}.gold.{tbl}").collect()[0].cnt
    print(f"  {tbl:35s} {cnt:>10,} rows")
```

========== GOLD LAYER SUMMARY — Run ID: 7549bffc ========== dim_date 1,826 rows dim_artist 275 rows dim_album 347 rows dim_genre 25 rows dim_media_type 5 rows dim_employee 8 rows dim_track 3,503 rows dim_customer 62 rows fact_sales 2,240 rows fact_sales_customer_agg 59 rows

```
%md
## 13. Dimensional Integrity Checks
```

13. Dimensional Integrity Checks

```
# Check: every fact row joins to a customer dimension
orphan_customers = spark.sql(f"""
    SELECT COUNT(*) as orphans
    FROM {catalog}.gold.fact_sales fs
    LEFT JOIN {catalog}.gold.dim_customer dc ON fs.customer_key = dc.customer_key
    WHERE dc.customer_key IS NULL
""").collect()[0].orphans
print(f"Orphan customer keys in fact_sales: {orphan_customers}")

# Check: aggregate fact totals match fact_sales rollup
agg_total = spark.sql(f"SELECT SUM(total_sales_amount) as total FROM
{catalog}.gold.fact_sales_customer_agg").collect()[0].total
fact_total = spark.sql(f"SELECT SUM(line_amount) as total FROM {catalog}.gold.fact_sales").collect()[0].total
print(f"fact_sales total:          {fact_total}")
print(f"fact_sales_cust_agg total: {agg_total}")
print(f"Match: {'✓' if abs(float(fact_total or 0) - float(agg_total or 0)) < 0.01 else '✗'}")
```

Orphan customer keys in fact_sales: 0 fact_sales total: 2328.60 fact_sales_cust_agg total: 2328.60 Match: ✓

```
# Sample fact_sales
display(spark.sql(f"SELECT * FROM {catalog}.gold.fact_sales LIMIT 10"))
```

Table

```
# Sample aggregate — top customers by sales
display(spark.sql(f"SELECT * FROM {catalog}.gold.fact_sales_customer_agg ORDER BY total_sales_amount DESC LIMIT
10"))
```

Table